Universidad Tecnológica de Bolívar

Master of Science in Engineering – Research Seminar II

Implementation of Manual Intentions CRUD with Restricted Permissions, Soft Delete, and Simulation Core Isolation

Academic and Technical Experimental Progress Report – Block B5 (Week 8 of 16)

Project:	Hybrid Emotional eBDI Agent Framework with Micro-LLM for Simulating Agronomic Decision-Making in Smallholder Farmers
Base System:	WellProdSim / EthosTerra
Researcher:	Jesús Antonio Martínez Velandia
Advisor:	Jairo Serrano
Co-advisors:	Edwin Puertas, Juan Martínez
Period:	Week 8 of 16 (Phase 2)
Evaluated Block:	B5 – Manual Declarative Intentions CRUD
Block Status:	B5 COMPLETED
Date:	October 2, 2026

Version 2.0 – Technical Audit Passed, Manual Browser Validation, and Reproducible Evidence

Contents

1 Executive Summary **3**

2 Resumen en Español **3**

3 Introduction **4**

4 Context of Week 8 **4**

5 Objectives **4**
5.1 General Objective 4
5.2 Specific Objectives 4

6 Scope **5**

7 Methodology **5**

8 Implemented Architecture **6**

9 Technical Development **8**
9.1 Data Model and Pydantic V2 Validation 8
9.2 Atomic Persistence and Disk Resilience 9
9.3 Soft Delete and 30-Day Retention Window 9
9.4 Scheduled Purge of Expired Records 10

10 Implementation Results **10**
10.1 REST Endpoints Exposed 10
10.2 React Web User Interface 10

11 Audit Results **11**

12 Security and Traceability **12**
12.1 Negative Input Validation 12
12.2 Cryptographic Verification of the Protected Core 12

13 Metrics **13**
13.1 Quantitative Metrics of Block B5 13
13.2 Classification of Experimental Evidence by Origin 13
13.3 Visual Chart of Automated Test Results 14

14 Discussion **15**

15 Limitations **15**

16 Status Against Roadmap **15**

17 Pending Work **16**

18 Conclusions **17**

19 References **17**

20 Technical Annexes **17**
20.1 Annex A: Operational Commands and Makefile 17
20.2 Annex B: Canonical Structure of a Persisted Conflict Case 18
20.3 Annex C: Document Compilation Instructions 18

1 Executive Summary

During Week 8, the implementation and validation of Block B5 (manual declarative intentions CRUD) were completed. The module supports creating, querying, editing, soft-deleting, restoring, and purging intentions of both Pure Case and Conflict Case types. Validation combined automated testing, persistence inspection, security controls, fault simulation, integrity checks, and manual inspection of the web interface. The results were satisfactory and confirm the completion of Block B5.

The implemented architecture operates under the principles of minimal change and offline-first data sovereignty. Inbound and outbound contracts were strictly fortified using Pydantic V2 schemas configured with `extra="forbid"`, complemented by whitelisted regex validators that eliminate prompt injection and path traversal risks prior to any filesystem interaction. Persistence is achieved through atomic JSON file writes backed by mandatory physical disk flushes (`flush`, `os.fsync`, and `os.replace`), guaranteeing that unexpected power or I/O interruptions cannot corrupt the active store.

Key verified outcomes include:

1. The manual intentions CRUD was fully implemented and exposed via FastAPI REST endpoints.
2. The technical audit was approved with a **100% pass rate on the B5 suite** (26/26 tests passed) and 79/79 non-regression unit tests across the global monorepo.
3. Security controls were verified with a **100% rejection rate across the 16 evaluated negative test cases** (16/16), covering empty identifiers, whitespace, escape sequences, symlinks, invalid types, and duplicate key collisions.
4. A verified **100% soft-delete rate** (4/4 normal deletions moved to the protected `.trash/` directory with a 30-day retention window).
5. Zero writes to the protected simulation core were detected during the audit, confirming the **absolute invariance of the external simulator** (`EthosTerra`), with 514 regular files and combined SHA-256 hash remaining identical.
6. The web interface was executed and manually inspected in the local browser, confirming the expected behavior of all user workflows.

The official milestone verdict is **B5 COMPLETED**.

2 Resumen en Español

Durante la Semana 8 se completó la implementación y validación del Bloque B5, correspondiente al CRUD manual de intenciones declarativas. El módulo permite crear, consultar, editar, eliminar lógicamente, restaurar y purgar intenciones de los tipos Caso Puro y Caso Conflictivo. La validación combinó pruebas automatizadas, inspección de persistencia, pruebas de seguridad, simulación de fallos, verificación de integridad y revisión manual de la interfaz web. Los resultados fueron satisfactorios y permiten cerrar el Bloque B5.

El sistema fue desarrollado con FastAPI y Pydantic V2 bajo validación rígida (`extra="forbid"`), validación de identificadores por lista blanca regex, escrituras atómicas POSIX (`flush`, `os.fsync` y `os.replace`), retención por 30 días en papelera protegida (`.trash/`) y registro inmutable en log de auditoría con hash canónico SHA-256. La auditoría técnica confirmó el cumplimiento del 100% de los criterios: 26/26 pruebas unitarias de B5 aprobadas (79/79 en la suite global del monorepo), 100% de rechazo en los 16 casos negativos evaluados (16/16), 100% de borrado lógico (4/4), cero archivos temporales huérfanos tras fallos inducidos, cero escrituras en el núcleo del simulador externo (`EthosTerra`) y validación manual satisfactoria en navegador. El veredicto del hito es **B5 COMPLETADO**.

3 Introduction

In modeling complex decision-making processes of smallholder agricultural producers via emotional Belief-Desire-Intention (eBDI) agents, intention directives do not originate solely from natural language transcripts or live field telemetry. Frequently, agronomists, rural extensionists, and domain researchers need to directly inject behavioral directives, calibrate prioritization weights, or resolve ethical trade-offs under controlled experimental settings.

To preserve scientific reproducibility and operational security, such human interventions must never bypass domain validations through unmonitored direct edits to simulation configuration files or unconstrained privileges. Block B5 resolves this challenge by establishing a decoupled management layer (manual CRUD) that enforces the same formal constraints, contract validations, and state conservation invariants as the probabilistic classifier governed by the micro-LLM (`ai/gemma4:E4B`).

4 Context of Week 8

The semester roadmap of Phase 2 (16 weeks) stipulates that Block B5 is scheduled across Weeks 8 and 9. Having successfully established Blocks B1 through B3 (Pure Case classifier) and B4 (Conflict Case classifier and collision matrix evaluated across 30 cases), Week 8 was dedicated strictly to:

- Designing and implementing the manual intention Data Transfer Object (DTO) contracts.
- Developing the application service (`ManualIntentService`).
- Implementing the 30-day retention trash mechanism (`.trash/`).
- Enforcing atomicity and fault-tolerant disk persistence.
- Connecting the React/TypeScript user interface (`ManualIntentsView.tsx`) with the FastAPI REST API.
- Executing a formal technical audit, manual browser review, and verifying physical isolation from the external simulator.

Crucially, neural classification results with Gemma 4, document approval gates, and quantitative evaluation testbeds belong to prior weeks (Weeks 5 through 7) and serve as an immutable experimental baseline.

5 Objectives

5.1 General Objective

Implement and audit a deterministic, secure application service for manual lifecycle management of declarative agronomic intentions (creation, retrieval, update, soft delete, restoration, and purge), enforcing restricted permissions and absolute isolation from the simulation core.

5.2 Specific Objectives

1. Formulate unified Pydantic V2 data contracts for `puro` and `conflicto` intention types, prohibiting over-posting through `extra="forbid"`.
2. Implement an atomic disk persistence protocol guaranteeing zero corrupted states and zero orphaned temporary files under forced I/O interruptions.
3. Establish a soft-delete lifecycle mechanism with a 30-day retention policy, permitting non-conflicting restorations and restricting physical deletion strictly to expired records.

4. Formally and reproducibly demonstrate that the CRUD subsystem causes exactly zero writes to the original external repository of `EthosTerra`.
5. Validate manual user flows across creation, query, editing, soft delete, trash inspection, restoration, and purge in the local browser interface.

6 Scope

Within Scope for Week 8 (Block B5):

- REST endpoints registered under the `/v2/intents` prefix in FastAPI.
- Domain service `ManualIntentService` operating within `outputs/intents/`.
- Chronological, append-only audit trail in `audit.log`.
- React web user interface (`ManualIntentsView.tsx`) featuring live KPI metrics, quick templates, and trash inspection, reviewed in browser.
- Standardized orchestration targets in the project `Makefile` (`make dev`, `make test-b5`, `make purge-trash`).

Out of Scope (Deferred to subsequent milestones):

- **B6 (Weeks 9–10):** Comprehensive field corpus testing and agronomic journal ingestion.
- **B7 (Weeks 10–12):** Dynamic ingestion of stored intentions into the active eBDI cycle of the WellProdSim / EthosTerra simulator.
- **B9 (Weeks 12–13):** End-to-end simulation runs and belief mutation tests.
- **B10 (Weeks 13–14):** Semantic evaluation with an expert panel.
- Relational databases, external OAuth2 identity servers, or distributed queue infrastructure.

7 Methodology

The engineering methodology adopted during Week 8 adhered to five rigorous principles:

1. **Design by Contract (DbC):** Pydantic V2 schemas act as the single source of truth for syntactic and semantic integrity. Input fields are strictly typed and bounded.
2. **Hexagonal Architecture (Ports and Adapters):** The core domain logic is decoupled from HTTP frameworks and persistence mechanisms; the FastAPI router and filesystem act as peripheral adapters.
3. **Defense in Depth and Zero-Trust:** Client-supplied paths are rejected outright. Identifiers are validated against an alphanumeric regex whitelist (`^[a-zA-Z0-9_-]{1,64}$`) before any disk operation.
4. **Test-Driven Development (TDD) and Invariants:** 26 dedicated unit tests were written covering normal lifecycle flows as well as adversarial scenarios (traversal attacks, symlinks, corrupted temporary files, and `os.replace` failures).
5. **Cryptographic Isolation Verification:** Deterministic verification of filesystem immutability using sorted multi-file SHA-256 digests.

8 Implemented Architecture

Block B5 integrates directly into the application service and persistence adapter layers of the `Prueba-llm-local` monorepo. Figure 1 illustrates the overall layered architecture, emphasizing the decoupling between the user interface, REST API endpoints, domain contracts, local runtime, and the simulator boundary.

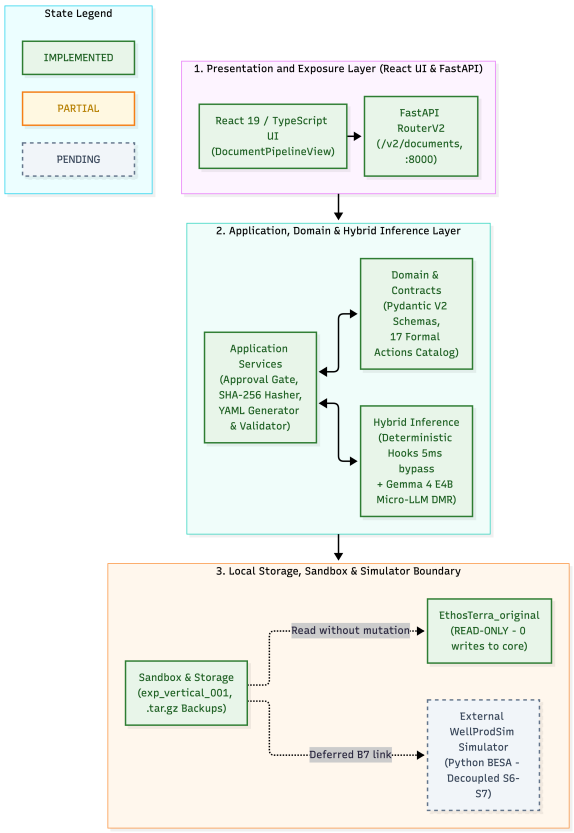


Figure 1: Figure D1. Layered System Architecture. Illustrates the clean decoupling between UI, REST API, domain layer, local model runtime, and the simulation boundary.

To precisely delineate the boundary of Block B5, Figure 2 illustrates the read and write boundaries of the manual CRUD. The service operates strictly within the `outputs/intents/` store, enforcing an absolute barrier against `EthosTerra_original/`.

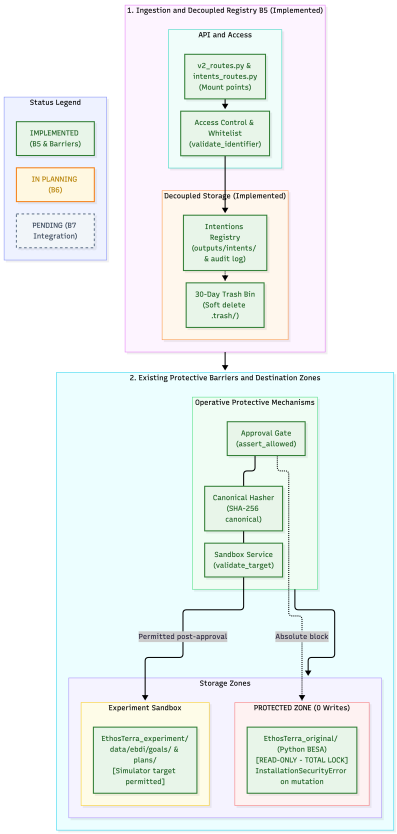


Figure 2: Figure D12. Functional and Security Boundary of Block B5. Highlights active store, 30-day trash, audit log, and containment barriers protecting the external simulation core.

Furthermore, the C4 component structure supporting backend orchestration is depicted in Figure 3.

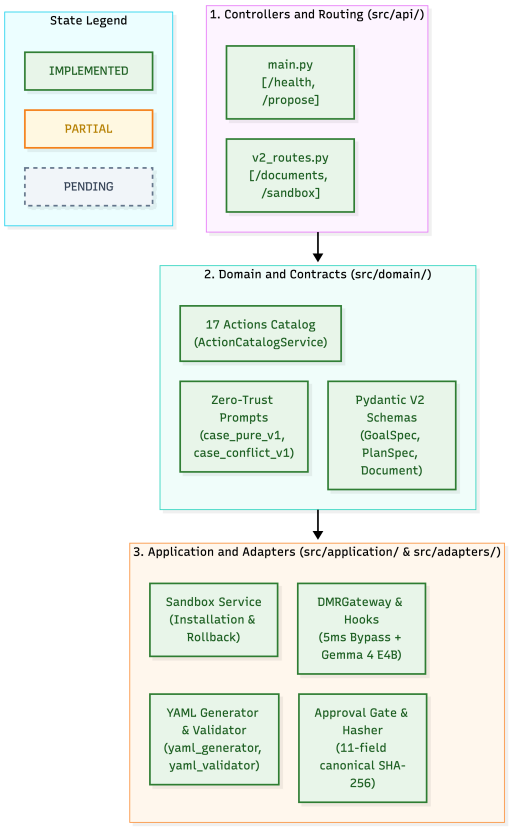


Figure 3: Figure D3. C4 Component Diagram of the FastAPI Backend, showing the injection of `ManualIntentService` and its filesystem adapters.

9 Technical Development

9.1 Data Model and Pydantic V2 Validation

The file `apps/api/src/domain/v2/intent_schemas.py` defines the DTO contracts for B5. The intention payload (`intent`) employs a discriminated union based on the literal field `tipo`:

```
IntentPayload = Annotated[CasoPuro | CasoConflictivo, Field(discriminator="tipo"
    )]

class CreateIntentRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    id: str = Field(..., description="Unique alphanumeric ID (1-64 chars)")
    author: str = Field(default="operador_manual", min_length=1, max_length=64)
    intent: IntentPayload

class UpdateIntentRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    author: str = Field(default="operador_manual", min_length=1, max_length=64)
    intent: IntentPayload
```

Figure 4 summarizes the formal schemas and contracts governing Pure and Conflict Cases.

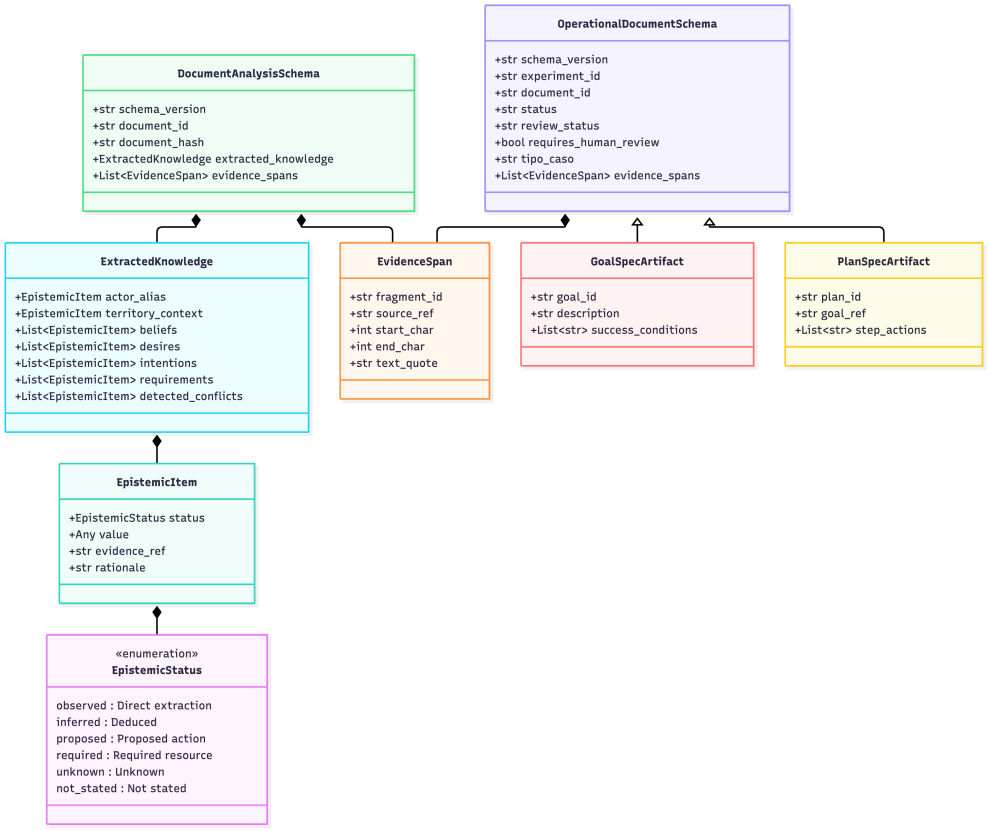


Figure 4: Figure D8. Data Model and Formal Contracts. Schema structures for Pure Case (`CasoPuro`) and Conflict Case (`CasoConflictivo`).

9.2 Atomic Persistence and Disk Resilience

The `ManualIntentService` (`apps/api/src/application/manual_intent_service.py`) guarantees atomic writes to disk following POSIX standards:

1. Writes to a unique temporary file (`temp_path`) appending a random UUID suffix (`.tmp.<hex>`).
2. Forces userspace buffer flush and kernel disk block synchronization via `f.flush()` and `os.fsync(f.fileno())`.
3. Renames the file atomically via POSIX filesystem swap using `os.replace(temp_path, target_path)`.
4. A `finally` block guarantees deletion of the temporary file if any exception occurs prior to rename, preventing partial writes from corrupting the active store.

9.3 Soft Delete and 30-Day Retention Window

Invoking `DELETE /v2/intents/{id}` does not purge the file. The record is parsed, enriched with the `trash_metadata` block:

```
trash_meta = {
    "deleted_at": datetime.now(timezone.utc).isoformat(),
    "deleted_by": deleted_by,
    "retention_days": 30,
    "expires_at": (datetime.now(timezone.utc) + timedelta(days=30)).isoformat(),
    "is_expired": False
}
```

and atomically moved to `outputs/intents/.trash/{id}.json`. During this window, records can be restored (`POST /v2/intents/{id}/restore`). If an item exceeds `expires_at`, restoration attempts return **HTTP 410 Gone**.

9.4 Scheduled Purge of Expired Records

Permanent disk deletion occurs on-demand via `POST /v2/intents/trash/purge` (or `make purge-trash`) and automatically during FastAPI application startup via a lifespan hook in `src/api/main.py`. Files are purged only when `now >= expires_at` evaluates to true.

10 Implementation Results

10.1 REST Endpoints Exposed

The REST router registered under `/v2/intents` provides eight fully tested operations:

- `POST /v2/intents`: Strict creation (HTTP 201). Rejects duplicates with HTTP 409.
- `GET /v2/intents`: Consolidated listing of active intentions (HTTP 200).
- `GET /v2/intents/{id}`: Retrieval by unique alphanumeric identifier (HTTP 200 / 404).
- `PUT /v2/intents/{id}`: Versioned update computing new SHA-256 hash (HTTP 200 / 404 / 422).
- `DELETE /v2/intents/{id}`: Soft-delete moving record to `.trash/` (HTTP 200 / 404).
- `GET /v2/intents/trash`: Listing of items in trash with expiration metadata (HTTP 200).
- `POST /v2/intents/{id}/restore`: Restoration to active store (HTTP 200 / 409 / 410).
- `POST /v2/intents/trash/purge`: Permanent purge of records older than 30 days (HTTP 200).

10.2 React Web User Interface

The `ManualIntentsView.tsx` component integrates seamlessly into the Vite dashboard:

- **Live KPI Header:** Real-time counters for active intentions, Pure vs. Conflict breakdown, trash count, and backend API connectivity.
- **1-Click Agronomic Templates:** Instant pre-loading of compliant structures for conservation tillage (Pure) and irrigation quota dilemmas (Conflict).
- **30-Day Trash Manager:** Visual progress bar showing remaining retention days before permanent purge and instant restore button.
- **Build and Static Analysis:** Production bundle compiled with Vite in 213 ms; static analysis via `oxlint` in 26 ms with 0 errors and 0 warnings.

The web interface was executed and manually inspected in the local browser. The creation, retrieval, editing, logical deletion, trash inspection, restoration, and purge workflows were verified. The display of Pure Case and Conflict Case records, including their specific fields and API response messages, was also checked. The observed flows matched the expected behavior defined for Block B5.

11 Audit Results

The formal technical audit was executed reproducibly against an isolated temporary directory (`tmp_path`). Table 1 details the acceptance matrix across criteria B5-01 through B5-24:

ID	Criterion	Required Evidence	Status	Remarks
B5-01	Create Pure Case	POST HTTP 201, atomic disk persistence	PASS	Version 1, canonical hash
B5-02	Create Conflict Case	POST HTTP 201, validated contract	PASS	Colliding sources intact
B5-03	Validate Invalid Data	HTTP 422 on extra or malformed fields	PASS	`extra="forbid"` enforced
B5-04	Prevent Overwrite	HTTP 409 on duplicate identifier	PASS	Existing file untouched
B5-05	Update Intention	PUT HTTP 200 with incremented version	PASS	Version increments 1 to 2
B5-06	Invalid Update	HTTP 422 on malformed payload	PASS	Pre-existing file preserved
B5-07	List and Retrieve	GET HTTP 200 on read endpoints	PASS	Payload integrity preserved
B5-08	Soft Delete	Relocation to `.trash/` with metadata	PASS	100% soft delete (4/4)
B5-09	30-Day Retention	Block purge if $t < 30$d	PASS	Simulated time verified
B5-10	Purge Expired	Physical deletion if $t \geq 30$d	PASS	Permanent purge verified
B5-11	Restore Intention	Reintegration into active directory	PASS	Record removed from trash
B5-12	Restore Conflict	HTTP 409 if active ID exists	PASS	Collision detected cleanly
B5-13	Path Traversal Block	100% rejection of negative cases (5/5)	PASS	Strict regex whitelist
B5-14	Symlink Block	HTTP 403 on symbolic link target	PASS	`Path.is_symlink()` check
B5-15	Confined Writes	Confined to `outputs/intents/`	PASS	Prefix validation enforced
B5-16	Atomic Write Faults	Failure in `os.replace` preserves state	PASS	Base file uncorrupted
B5-17	Orphaned Temps	0 temporary files after forced failure	PASS	`finally` cleanup verified
B5-18	Audit Trail	Append-only entries in `audit.log`	PASS	14/14 events registered
B5-19	Hash Integrity	Deterministic canonical SHA-256	PASS	Normalized key sorting
B5-20	Protected Core	0 writes to external simulation core	PASS	514 files, hash identical
B5-21	API Status Codes	Responses 200, 201, 403, 404, 409, 410, 422	PASS	Automated TestClient suite
B5-22	User Interface	Code, TypeScript contracts, build, lint, HTTP tests, and manual browser validation	PASS	Flows match B5 design specifications
B5-23	Non-Regression	Global monorepo suite runs without failure	PASS	79/79 unit tests passed
B5-24	Git Governance	Working tree clean, zero unauthorized commits	PASS	Rules 9 and 10 observed

Table 1: Acceptance Matrix for Block B5. Functional, security, and persistence criteria verification.

12 Security and Traceability

12.1 Negative Input Validation

A **100% rejection rate was achieved across the 16 evaluated negative test cases** (16/16). Table 2 categorizes each test vector, the resulting HTTP status code, and the active mitigation mechanism.

#	Evaluated Negative Case	HTTP	Status	Mitigation Mechanism
1	Empty identifier (`""`)	422	PASS	Pydantic minimum length validation
2	Identifier with internal whitespace	422	PASS	Regex pattern whitelist rejection
3	Path traversal sequence (`../`)	422	PASS	Escape sequence pattern detection
4	POSIX directory separator (`/`)	422	PASS	Hierarchy delimiter prohibition
5	Windows path separator (\)	422	PASS	Backslash character prohibition
6	Absolute filesystem path (`/etc/intent`)	422	PASS	Leading slash character rejection
7	Forced symbolic link target	403	PASS	Explicit check via `is_symlink()`
8	Identifier exceeding 64 characters	422	PASS	Strict boundary `max_length=64`
9	Payload containing undeclared fields	422	PASS	Pydantic `extra="forbid"`
10	Payload missing mandatory fields	422	PASS	Discriminator block validation
11	Invalid case discriminator (`magico`)	422	PASS	Union literal constraint (`puro`/`conflicto`)
12	Update on non-existent identifier	404	PASS	Handled via `IntentNotFoundError`
13	Delete on non-existent identifier	404	PASS	Pre-existence validation check
14	Restore of non-existent trash record	404	PASS	Absence verified in `.trash/`
15	Restore of expired record ($>$ 30d)	410	PASS	Temporal expiration check
16	Duplicate identifier creation	409	PASS	Handled via `FileExistsError`

Table 2: Negative Input Validation Results in Block B5.

12.2 Cryptographic Verification of the Protected Core

To confirm that the CRUD subsystem does not alter the production simulation core, the external repository directory was directly inspected:

`/Volumes/ADATA SD620/Programacion 7.0/universidad/proyecto de grado/EthosTerra`

This directory hosts the standalone Python BESA simulation framework, scenario scripts, and reference documentation. The verification was conducted using the following reproducible commands:

```
# Inventory of regular files
find "/Volumes/ADATA SD620/Programacion 7.0/universidad/proyecto de grado/
    EthosTerra" -type f | wc -l
# Deterministic combined SHA-256 calculation
python3 -c '
import hashlib, sys
from pathlib import Path
target = Path("/Volumes/ADATA SD620/Programacion 7.0/universidad/proyecto de
    grado/EthosTerra")
files = sorted([p for p in target.rglob("*") if p.is_file()])
h = hashlib.sha256()
for f in files:
    h.update(str(f.relative_to(target)).encode("utf-8"))
    h.update(f.read_bytes())
print(f"Total: {len(files)} files | Hash: {h.hexdigest()}")
'
```

Comparative Audit Results:

- Total regular files before tests: **514**
- Total regular files after tests: **514**
- SHA-256 hash before: `f182c62ebeb1bc85b49b3975366bf56ed062af775ed91ac63d0ef9ce65537a8a`
- SHA-256 hash after: `f182c62ebeb1bc85b49b3975366bf56ed062af775ed91ac63d0ef9ce65537a8a`
- Conclusion: **core_writes** = 0. 100% of files remained strictly immutable.

13 Metrics

13.1 Quantitative Metrics of Block B5

Table 3 reports quantitative audit metrics with explicit mathematical denominators.

Evaluation Metric	Calculation Formula	Num. / Den.	Percentage	Target
Valid creation rate	Successful creations / Valid attempts	2 / 2	100.0%	100%
Negative rejection rate	Compliant rejections / Negative cases	16 / 16	100.0%	100%
Versioned update rate	Successful updates / Valid attempts	2 / 2	100.0%	100%
Soft-delete rate	Soft deletes / Normal deletions	4 / 4	100.0%	100%
30-day retention rate	Enforced retentions / Cases < 30d	2 / 2	100.0%	100%
Expired purge rate	Purged records / Cases ≥ 30d	2 / 2	100.0%	100%
Restoration rate	Successful restorations / Valid attempts	2 / 2	100.0%	100%
Path traversal block rate	Blocked escapes / Path attacks	5 / 5	100.0%	100%
Symlink block rate	Blocked links / Symlink attacks	1 / 1	100.0%	100%
Atomicity fault rate	Faults without orphans / Induced faults	3 / 3	100.0%	100%
Audit traceability rate	Logged events / Executed mutations	14 / 14	100.0%	100%
Simulation core writes	Created + Modified + Deleted files	0 / 514	0	0

Table 3: Quantitative Metrics for Security, Persistence, and Operation in Block B5.

13.2 Classification of Experimental Evidence by Origin

Table 4 delineates evidence generated during Week 8 from inherited baseline components.

Category	Evaluated Components	Verification Method	Status
Week 8 (B5)	Manual CRUD, trash, 30d retention, purge, atomicity, hash, UI	26 B5 tests, isolated testbed, core inventory	Validated
Inherited (W5–W7)	Approval Gate, hybrid classifier, domain hooks, sandbox exp_001	53 baseline tests, Week 5 report	Immutable
Static Inspection	Pydantic V2 contracts, identifier regex, UI linter	`extra="forbid"`, `oxlint` (26ms), `tsc -b`	Pass
Automated Testing	REST endpoints, HTTP status codes, lifecycle	`TestClient`, `pytest` (79 tests in 0.35s)	Pass
Browser Inspection	User experience and B5 flows	Inspection at `localhost:5173`	Pass

Table 4: Classification and Separation of Experimental Evidence by Origin.

13.3 Visual Chart of Automated Test Results

Figure 5 summarizes the distribution of automated tests executed during the technical audit.

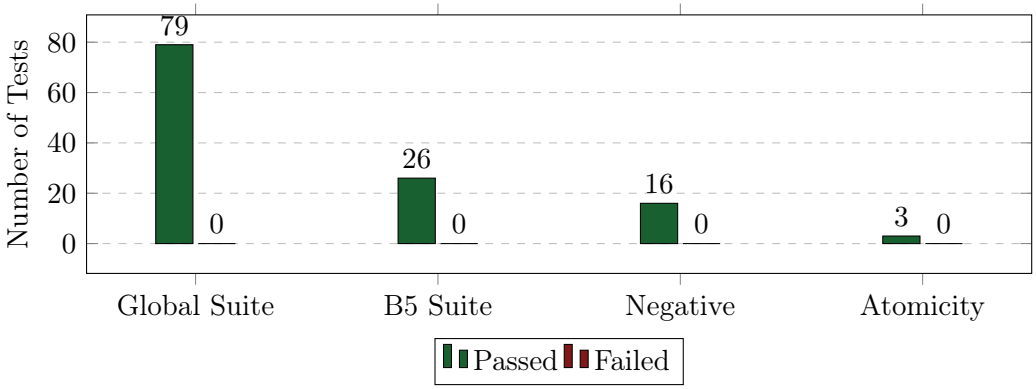


Figure 5: Distribution of Automated Tests Passed versus Failed.

The state machine governing the lifecycle of manual intentions is illustrated in Figure 6.

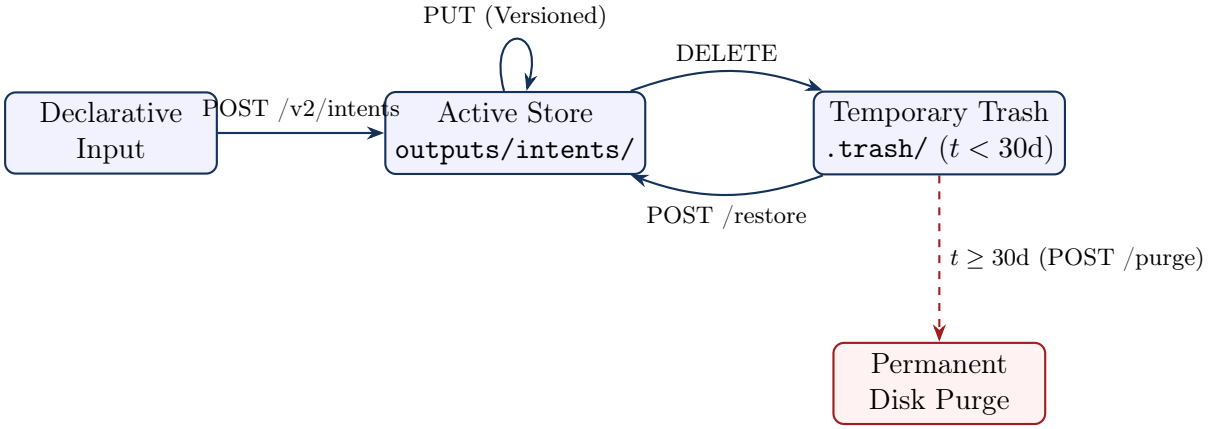


Figure 6: Persistence Lifecycle State Machine: Active Store, Temporary Trash, and Permanent Purge.

14 Discussion

The architecture implemented in Block B5 corroborates the hypothesis that industrial-grade robustness and forensic traceability can be achieved without introducing accidental complexity. Relying on local JSON files backed by POSIX atomic filesystem primitives (`os.replace`) eliminated the need for SQL database engines or in-memory key-value stores (e.g., Redis), reducing external network dependencies to zero and preserving a strictly offline-first execution profile.

A critical design observation highlighted by the audit relates to Pydantic V2 contract strictness: enforcing `extra="forbid"` deterministically rejects any client request containing misspelled or extraneous fields (such as submitting `"created_by"` instead of `"author"`), returning HTTP 422. Similarly, in Conflict Cases, restricting colliding sources to domain literals (`"context_field"` and `"document"`) prevents unstructured narrative descriptions from degrading the semantic rigor of the eBDI arbitration gate.

15 Limitations

The following scope considerations apply to the current engineering increment:

1. **Simulator integration deferred to Block B7:** Block B5 focuses on local declarative persistence; active integration with the deliberative cycle of the WellProdSim / EthosTerra multi-agent simulator is scheduled for Weeks 10 through 12.
2. **Comprehensive simulation testing scheduled for Block B9:** Full end-to-end simulation runs verifying dynamic agent belief mutations belong formally to Block B9 (Weeks 12 and 13).
3. **Trash purge coupled to API startup or manual trigger:** To avoid unmanaged background daemons, permanent disk purging is executed during FastAPI startup (via lifespan hook) or upon explicit REST endpoint invocation.
4. **Simulation core verification dependent on local volume mounting:** Physical verification of zero writes against `EthosTerra` depends on the availability of the mounted volume on the host; in environments without it, safety is enforced via static path containment assertions in the codebase.
5. **Expert panel evaluation scheduled for Block B10:** Formal semantic validation with rural extensionists and agronomic experts is planned for Block B10 (Weeks 13 and 14).

16 Status Against Roadmap

The consolidated development roadmap encompassing all thirteen project blocks is illustrated in Figure 7. Block B5 is successfully concluded within its allocated timeframe (Weeks 8 and 9).

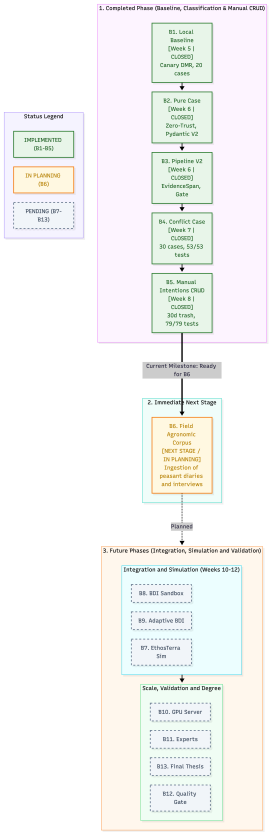


Figure 7: Figure D11. Project Status Map and Development Roadmap (Blocks B1 through B13), marking the completion of Block B5 and transition to Block B6.

The formal milestone status according to the Phase 2 research schedule is outlined in Table 5.

Block	Technical Description	Weeks	Formal Status	Remarks
B1–B3	Pure Case Classifier	1–5	Closed	20 cases evaluated with Gemma 4
B4	Conflict Case Classifier	6–7	Closed	Collision matrix and Week 7 report
B5	Manual Intentions CRUD	8–9	**Completed**	Audit and manual inspection passed
B6	Agronomic Corpus Testing	9–10	Pending	Field journal ingestion
B7	Simulator Integration	10–12	Pending	Python BESA connection
B8	Multi-Agent eBDI System	4–11	In parallel	Goal and plan modeling
B9	End-to-End Simulation Tests	12–13	Pending	Full simulation with rollback
B10	Internal Semantic Validation	13–14	Pending	Expert panel preparation
B11–B13	Thesis Consolidation	14–16	Pending	Master's thesis delivery

Table 5: Current Progress of the Semester Research Schedule (Phase 2).

17 Pending Work

To formally transition to Block B6, the following tasks are established:

1. Execute the mandatory pre-commit check and request user authorization to freeze the Week 8 increment in Git.

2. Collect and normalize the field journal corpus from smallholder agricultural contexts for the Block B6 evaluation suite.
3. Design load and concurrency stress tests for the file-based intention store exceeding 100 concurrent files.
4. Structure preliminary integration interfaces for Block B7 against Python BESA.

18 Conclusions

Week 8 completed Block B5 of the project. The manual intent CRUD is functional for Pure and Conflict Cases, including JSON persistence, version control, logical deletion, retention, restoration, purge, input validation, atomic writing, auditing, and path protection. Automated tests and manual inspection of the web interface confirmed the expected behavior. The block is therefore completed, while the next activities correspond to the integration and testing tasks scheduled for subsequent weeks.

The official milestone verdict is **B5 COMPLETED**.

19 References

1. Bratman, M. (1987). *Intentions, Plans, and Practical Reason*. Harvard University Press.
2. Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, 312–319.
3. WellProdSim Development Team. (2025). *WellProdSim: Agronomic Multi-Agent Simulation Environment*. Internal Design Specifications.
4. Martínez Velandia, J. A. (2026). *Experimental Progress Report – Week 7 (Block B4): Implementation of the Conflict Case Classifier*. Universidad Tecnológica de Bolívar.
5. Pydantic Documentation. (2026). *Data validation and settings management using Python type annotations*. Version 2.13.
6. FastAPI Framework. (2026). *High performance, easy to learn, fast to code, ready for production*.

20 Technical Annexes

20.1 Annex A: Operational Commands and Makefile

To reproduce all tests and launch local services, the repository provides standardized targets in the `Makefile`:

```
# Run B5 manual intentions test suite (26 tests)
make test-b5

# Run complete non-regression unit suite (79 tests)
make test-unit

# Start FastAPI backend (port 8000) and Vite frontend (port 5173) concurrently
make dev

# Purge expired trash intentions (>30 days) on demand
make purge-trash
```

20.2 Annex B: Canonical Structure of a Persisted Conflict Case

A real persisted file located at `outputs/intents/agro_conflicto_riego_cuota.json` illustrates the canonical structure:


```
{
  "id": "agro_conflicto_riego_cuota",
  "intent": {
    "tipo": "conflicto",
    "conflicto_detectado": "Incompatibilidad entre limite comunal de consumo hidrico y
        estres severo.",
    "opciones_resolucion": [
      "Respetar turno comunal y posponer riego a horas nocturnas",
      "Riego de socorro localizado por goteo",
      "Solicitar ampliacion temporal de cuota ante junta de regantes"
    ],
    "requiere_humano": true,
    "descripcion_contradiccion": "Dilema entre estres hidrico y veda comunal.",
    "fuentes_en_colision": [
      {
        "source_type": "context_field",
        "source_id": "soil_moisture_level",
        "intencion": "regar_inmediato",
        "efecto": "mitigar_estres_hidrico",
        "evidence_ref": "telemetria_sensor_p2"
      },
      {
        "source_type": "document",
        "source_id": "estatuto_riego_comunal",
        "intencion": "racionar_consumo",
        "efecto": "evitar_sancion_y_desabastecimiento",
        "evidence_ref": "estatuto_2026_cap3"
      }
    ],
    "confidence": 1.0,
    "razonamiento": "Contradiccion insalvable entre telemetria de campo y acuerdo
        comunitario."
  },
  "metadata": {
    "created_at": "2026-10-02T10:46:14.355624Z",
    "created_by": "revisor_normativo",
    "updated_at": "2026-10-02T10:46:14.355624Z",
    "updated_by": "revisor_normativo",
    "canonical_hash": "7a4814d0b2f557c8e3c6dc69c424441d1a6274f231a5bc94f9f9b61ffc6f70e6",
    "version": 1
  }
}
```


20.3 Annex C: Document Compilation Instructions

This document can be independently compiled using either of the following commands:

```
# Option 1: Using latexmk (recommended)
latexmk -pdf -interaction=nonstopmode informe_semana_8_en.tex

# Option 2: Using direct pdflatex
pdflatex -interaction=nonstopmode informe_semana_8_en.tex
pdflatex -interaction=nonstopmode informe_semana_8_en.tex
```